How Our System Works

What the project builds, how it works, and how it compares, explained simply

Mihir Bhati (24BCE5341) · Ahana Sharan (24BCE5334)

25 October 2026 · Project folder: D:\MyClaudeProject\nlp

The project in one sentence. Our framework takes a description of any new app and automatically designs, tests and delivers the best way for a ready-made AI model to understand that app's requests. We compared it with a generic template, a person, random choices and a trained published model.

Contents

1. The problem: turning a sentence into something a program can act on
2. Two words to keep apart: intents and designs
3. "If we already have the answers, why do we need the project?"
4. Two ways to build the understanding part: train a model, or give instructions
5. XLM-R explained: what it is and what it does in our project
6. qwen explained: the AI our framework uses
7. The "note" (prompt): what it is and how it is made
8. The 12 designs: 12 ways of writing the note
9. What our framework does, step by step (with the real run)
10. Once the pipeline is running: what happens to one request
11. Testing is not training
12. The exam: how our system is compared with XLM-R and the others
13. All the systems we compared, and the results
14. Is it innovative? An honest answer
15. Why it chooses from 12 designs, and what comes next
16. What to say: the whole system in a few sentences

1. The problem: turning a sentence into something a program can act on

Imagine a home-assistant app. A user types:

> "turn off the bedroom lights"

A computer program cannot act on a sentence. It needs something precise:

- **intent** = `iot_hue_lightoff`: **what** the user wants to do
- **entity** `house_place` = `bedroom`: the **details** needed to do it

Turning a sentence into that precise answer is called **language understanding**. The set of steps that does it is the **NLP pipeline**: text goes in, a precise answer comes out.

```
"turn off the bedroom lights"  ->  [ NLP pipeline ]  ->  intent = iot_hue_lightoff,
house_place = bedroom
```

Every app needs its own pipeline, because every app has different requests. A hospital app needs intents such as `book_appointment` and `cancel_appointment`; a home assistant needs `alarm_set` and `iot_hue_lightoff`.

2. Two words to keep apart: intents and designs

In project P1 there happen to be 12 of each, which is confusing. They are completely different things:

	What it is	How many	Examples
Intents	the possible answers: the types of request the app understands	different for each project: 12 for P1, 10 for P2	set alarm, remove alarm, lights on, lights off, dim lights, start coffee machine...
Designs	the ways of building the pipeline (the ways of writing the note, section 8)	always 12, for every project	joint-fixed-llm, schema-sim-llm, twostage-sim-tfidf...

Our framework's job: for a given project, pick **the best of the 12 designs**, so the AI correctly identifies which of that project's **intents** each new sentence is.

3. "If we already have the answers, why do we need the project?"

We have answers for only **a small number of sentences**, not for everything users will ever say. For the home assistant, the MASSIVE dataset gives us:

- **120 example sentences** with their correct answers (10 for each intent)
- **100 practice sentences** with their correct answers (development data)

- **300 exam sentences** with their correct answers (test data, kept locked)

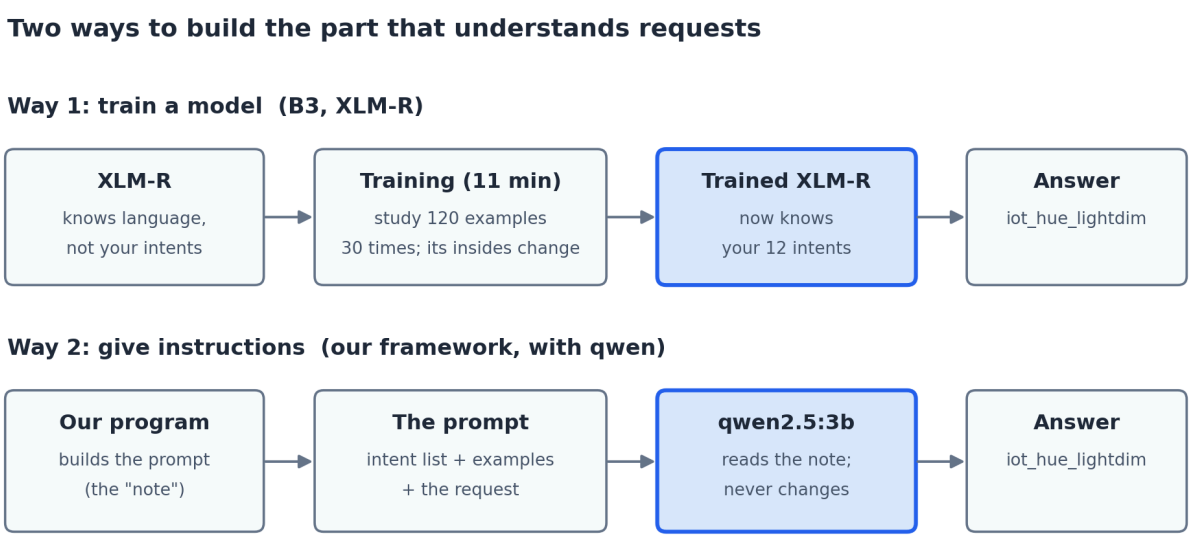
But a real app receives **new sentences every day that nobody has answered**: "kill the lights", "I'm going to bed, lights off", "can you make it less bright in here". Nobody can write the answer for every possible sentence, so the app must work it out itself.

The sentences with answers are used for only two things: as **examples** that show the AI what kind of answer we want, and as **test questions** to check how often the system gets new sentences right (we hide the answers from it and compare afterwards). Like school: the teacher has the answer key for the practice test, but the point is whether the student can answer **new** questions later.

For each labelled sentence the dataset gives two things: the **intent** ("make a room darker" is iot_hue_lightdim) and the **details** ("room" is a house_place). People labelled them by hand.

In a real project such as a hospital app, the developer writes the list of request types and about 10 example sentences for each, perhaps an hour's work. Our framework then builds a system that handles thousands of new patient messages automatically.

4. Two ways to build the understanding part



Way 1 changes the model. Way 2 changes only the text sent to it.

An everyday picture: a new waiter. The waiter speaks English perfectly but does not know your restaurant's menu. A customer says "I'd like something hot to drink", and the waiter has to work out which menu item that is.

	Way 1: memorise the menu	Way 2: hand the waiter a note
What happens	before starting work, the waiter studies the menu again and again until they know it by heart	every time a customer speaks, you hand the waiter a note: "these are the only things on our menu... example: 'something warm with milk' means coffee... what does this customer want?"
The waiter's brain	**changes**: this is training	**never changes**: they read the note, answer, and forget it
In our project	**XLM-R** (system B3)	**qwen**, with the note written by **our framework**

> **XLM-R memorises the answers. qwen reads a note each time. Our framework's job is to find the best note.**

5. XLM-R explained: what it is and what it does in our project

What it is. XLM-R is a **language model**: a program that has read huge amounts of text in about 100 languages and learned how words relate to each other. It is **not a chatbot**: it cannot chat or write answers. It was made by Meta (Facebook) researchers, and the creators of the MASSIVE dataset used it as their own reference system in their published paper.

What it knows, and what it does not. It knows English. It does **not** know our intents: it has never heard of `iot_hue_lightoff` or `book_appointment`. And it does **not** build pipelines; it cannot build anything.

What we do with it (system B3), step by step:

1. Load XLM-R. It knows language, but nothing about our app.
2. Add a small extra layer on top, with one output per intent (12 for P1).
3. **Train it** on our examples: show it the 120 labelled sentences 30 times. Each time it gets one wrong, its internal numbers are adjusted a little. This is called **fine-tuning**, and it took about 11 minutes.
4. After training, **it is the pipeline**: give it a sentence and it directly answers with the intent. No prompt, no instructions.

```
"make a room darker"  ->  [ trained XLM-R ]  ->  an intent
```

So what? Now it can sit the same exam as our system (section 12). XLM-R is not part of our framework: it is a **competitor**, showing how the standard, published approach (Way 1) does against ours (Way 2).

We trained two versions: **B3-k** on the same 10 examples per intent as every other system (the fair comparison), and **B3-full** on all the training data, 10 to 30 times more (only a reference, not a fair fight).

6. qwen explained: the AI our framework uses

qwen2.5:3b is a small version of something like ChatGPT, made by Alibaba. It runs on the laptop inside the **Ollama** app, so nothing is sent to the internet. "3b" means 3 billion internal numbers, small enough for the laptop's 4 GB graphics card.

- It is **already trained** and **never changes** in our project: nobody trains it.
- It understands language and follows instructions: give it a note and it answers.
- It plays **two roles**: an **advisor** while our framework designs the pipeline (section 9), and the **worker** that labels each request once the pipeline runs (section 10).

Our framework is not a model. It is a Python program (the code in the `src` folder). It does not understand language itself; it writes notes for qwen, sends them, and reads qwen's answers.

7. The "note" (prompt): what it is and how it is made

The note is the **prompt**: the block of text our program sends to qwen with each request. Here is the real note A2 uses on P1 for the request "make a room darker", produced by the project's own code:

```
Label this request and extract its information. Reply with JSON only.

Supported requests:
- alarm_query: Query alarms and their details.
- alarm_remove: Remove an existing alarm.
- alarm_set: Set a new alarm.
- iot_cleaning: Start the cleaning robot.
- iot_coffee: Start the coffee machine.
- iot_hue_lightchange: Change the color of a light.
- iot_hue_lightdim: Dim a light.
- iot_hue_lightoff: Turn off a light.
- iot_hue_lighton: Turn on a light.
- iot_hue_lightup: Increase the brightness of a light.
- iot_wemo_off: Turn off a smart device.
- iot_wemo_on: Turn on a smart device.
- none_of_the_above: the request is not one of the supported requests above

Information to extract, copied exactly as written in the request:
- alarm_type: Type of alarm (e.g., time, date)
- business_name: Business name for devices (e.g., coffee machine)
- business_type: Type of business (e.g., coffee machine, cleaning robot)
- change_amount: Amount to change (e.g., brightness, colour intensity)
- coffee_type: Type of coffee (e.g., espresso, cappuccino)
- color_type: Color type (e.g., RGB, HSL)
- date: Date for alarm (e.g., day, month, year)
- device_type: Type of device (e.g., light, socket, cleaning robot)
- event_name: Event name for devices (e.g., start, stop)
- general_frequency: General frequency for devices (e.g., daily, weekly)
- house_place: Place in the house (e.g., kitchen, bedroom)
- order_type: Order type for devices (e.g., on, off, dim, bright)
- place_name: Name of the place (e.g., living room, bathroom)
- player_setting: Setting for devices (e.g., volume, brightness)
- relation: Relation to other entities (e.g., off, on, dimmer)
- time: Time for alarm (e.g., hour, minute)
- time_zone: Time zone for time and date
- timeofday: Time of day for alarm (e.g., morning, evening)

Examples:
Request: turn room light off
Answer: {"intent": "iot_hue_lightoff", "entities": [{"type": "house_place", "text": "room"}]}
Request: make me happy juice
Answer: {"intent": "iot_coffee", "entities": [{"type": "coffee_type", "text": "happy
juice"}]}
Request: olly make the lights blue please
Answer: {"intent": "iot_hue_lightchange", "entities": [{"type": "color_type", "text":
"blue"}]}
Request: it's dark in the living room
Answer: {"intent": "iot_hue_lighton", "entities": [{"type": "house_place", "text": "living
room"}]}
Request: dim the lights at the living room
Answer: {"intent": "iot_hue_lightdim", "entities": [{"type": "house_place", "text": "living
room"}]}
Request: turn the lights off in the living room
Answer: {"intent": "iot_hue_lightoff", "entities": [{"type": "house_place", "text": "living
room"}]}
Request: olly make me coffee at eight am
Answer: {"intent": "iot_coffee", "entities": [{"type": "time", "text": "eight am"}]}
Request: turn the living room lights to one percent
Answer: {"intent": "iot_hue_lightchange", "entities": [{"type": "house_place", "text":
```

```
"living room"}, {"type": "change_amount", "text": "to one percent"}]}
Request: i can't see let's make it brighter
Answer: {"intent": "iot_hue_lightup", "entities": []}
Request: i would love a less bright up room
Answer: {"intent": "iot_hue_lightdim", "entities": []}
Request: i want the coffee machine to make me coffee
Answer: {"intent": "iot_coffee", "entities": [{"type": "device_type", "text": "coffee
machine"}]}
Request: modify the light colors of the house and make them dark instead
Answer: {"intent": "iot_hue_lightchange", "entities": [{"type": "color_type", "text":
"dark"}]}
Request: turn off my smart plug socket
Answer: {"intent": "iot_wemo_off", "entities": [{"type": "device_type", "text": "smart plug
socket"}]}
Request: brighten the lights on the front porch
Answer: {"intent": "iot_hue_lightup", "entities": [{"type": "house_place", "text": "front
porch"}]}
Request: put alarm on
Answer: {"intent": "alarm_set", "entities": []}
Request: remove dust with vacuum cleaner
Answer: {"intent": "iot_cleaning", "entities": [{"type": "device_type", "text": "vacuum
cleaner"}]}

Reply exactly: {"intent": "<one name>", "entities": [{"type": "<a type>", "text": "<text
copied exactly from the request>"}]}
Use an empty list if there is nothing to extract. Never invent a value.

Request: make a room darker
Answer:
```

It is long, but it is built from seven pieces, and **qwen wrote only one of them**:

#	Piece of the note	Where it comes from	Who made it
1	"Label this request and extract its information..."	fixed text in our code	written once, in our program
2	the list of intents (alarm_query, alarm_remove...)	the project's own intent list	the dataset
3	the short description after each intent ("Dim a light.")	the Analyzer step	**qwen**: the only piece it wrote
4	the list of fields and their descriptions	names from the project; descriptions from the Analyzer	names from the dataset; descriptions by **qwen**
5	the 16 examples ("turn room light off" -> iot_hue_lightoff...)	**real sentences from the dataset**, labelled by people	**picked** by our program; nobody wrote them
6	"Reply exactly: {...} ... Never invent a value."	fixed text in our code	written once, in our program
7	"Request: make a room darker"	the user	the user

qwen does not invent example sentences. The examples are real labelled sentences; our program only chooses which ones go on the note. The "similar" designs pick the 16 most similar to the request, which is why this note is full of light and room examples.

Not everything qwen writes is good. Look at piece 4: "color_type: Color type (e.g., RGB, HSL)" or "relation: ... (e.g., off, on, dimmer)". That is one reason the framework tests every design instead of trusting qwen's advice.

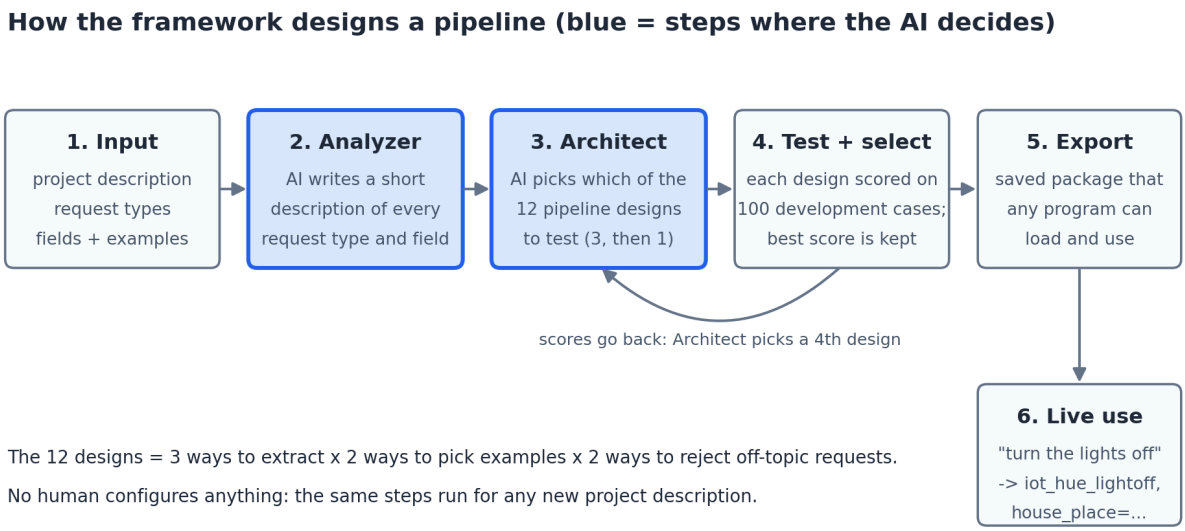
8. The 12 designs: 12 ways of writing the note

The designs use the same pieces, arranged differently. Each design makes three choices:

Choice	Options	What changes on the note
How to ask	joint (one question) · twostage (intent first, then details) · schema (one question, with descriptions)	schema adds pieces 3 and 4; twostage asks qwen twice
Which examples	fixed (the same one per intent, every time) · sim (the most similar to the request)	which real sentences go in piece 5
How to refuse off-topic requests	llm (qwen may answer none_of_the_above) · tfidf (refused before asking qwen if unlike every example)	whether none_of_the_above is on the intent list

3 × 2 × 2 = **12 designs**, named like `schema-sim-llm`. Every project has the same 12, whatever its intents.

9. What our framework does, step by step (with the real run)



The framework's steps. The blue steps are where qwen gives advice.

Testing every design costs time, so the agent tests **only 4 of the 12** and has to choose wisely which ones. This is exactly what happened on P1 in the real A2 run:

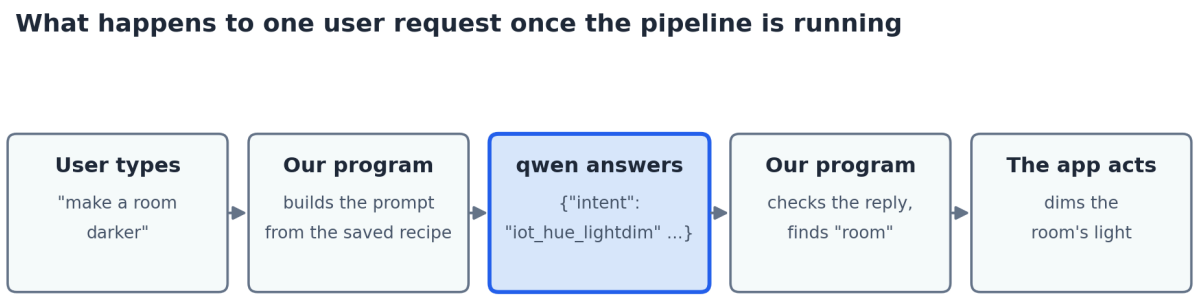
Step	Our program	qwen
1. Analyzer	"Write a short description of each intent and field."	writes them, e.g. "alarm_set: Set a new alarm."
2. Architect	"For this home assistant, which way of writing the note is best? Rank each choice."	**advises**: descriptions first; fixed examples better than similar ones; refuse off-topic with the cut-off
3. Test	builds 3 notes from that advice, and tests each on 100 practice sentences	**answers** the practice sentences with each note
	results: 0.845, **0.869**, 0.839	
4. Revise	"Here are the scores. Change one thing from the best note. What should it be?"	**advises**: let the model refuse off-topic itself
5. Test	builds that 4th note and tests it: **0.974**	answers the practice sentences
6. Select	keeps the best note (0.974) and saves it as the pipeline	nothing

"Advisor" means qwen is asked for its opinion on how to write its own note. Our program asks the questions, builds the notes, runs the tests, does the scoring and makes the final choice. qwen's first advice was partly wrong (fixed examples turned out worse); the testing caught that, and the best note won anyway.

System	Designs tested	How they are chosen
Our agent (A1, A2)	4	the AI reasons about which ones to try
B4 random search	4	at random
B4 exhaustive	all 12	tries everything: the best possible score

The original agent (A1) did not beat random choices. The improved agent (A2) searches in a more structured way and matched the best of all 12 on both projects while testing only 4.

10. Once the pipeline is running: what happens to one request



Every request a user types goes through these steps.

The saved pipeline is a **recipe**: which design, which examples, which descriptions. When a user types something, our program builds the note from the recipe, qwen answers, our program checks the answer and finds the details in the sentence, and the app acts on it. **The pipeline = our program's recipe + qwen doing the understanding.**

11. Testing is not training

	Training (XLM-R)	Testing notes (our framework)
What changes	**the model itself**: its internal numbers	**only the text** sent; qwen never changes
Does it remember?	yes, permanently: that is the point	**no**: qwen forgets after each request; the note is sent fresh every time
The practice sentences are for...	not used for learning (it learns from the 120 examples)	**comparing note designs**, to keep the one that gets the most right
Picture	a student studying until they memorise	open-book practice exams with 4 different note sheets; keep the sheet that scored best

In one line: our framework does not teach qwen anything. It measures which version of the note helps qwen answer best, and keeps that version. That is choosing, not training.

12. The exam: how our system is compared with XLM-R and the others

Both XLM-R and our system have **exactly the same job**: read a request and say its intent. So they sit **the same exam**: the same 80 requests, whose correct answers we know. Real answers on P1:

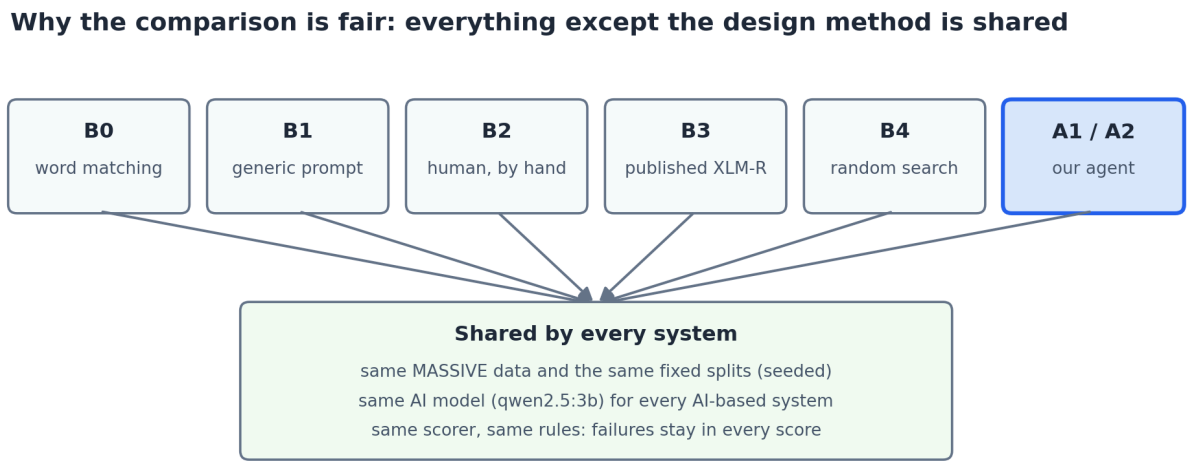
Request	Correct	XLM-R (B3-k) said	Our system (A2) said
turn the lights off please	iot_hue_lightoff	right	right
make a room darker	iot_hue_lightdim	wrong: iot_cleaning	right
switch on the lights	iot_hue_lighton	wrong: refused it	right
start up the coffee machine	iot_coffee	wrong: refused it	right
please remove any alarms that i have set	alarm_remove	wrong: refused it	right
start new smart socket	iot_wemo_on	right	wrong: refused it

The full count on 80 requests: both right on 68; only our system right on 8; only XLM-R right on 1. Turned into F1: **our system 0.974, XLM-R 0.909.**

Why XLM-R struggled: 10 examples per intent is very little to learn from, so it was often unsure and refused normal requests. qwen did not need to learn the intents: they are written on its note.

So what does the comparison tell us? For a new app with only about 10 examples per intent, giving a ready-made AI the right note (our way) got more answers right than training the published model (the standard way), and needed no training.

13. All the systems we compared, and the results



Every system shares the data, the AI model and the scorer.

System	What it is	In the waiter picture	P1 intent F1	P2 intent F1
B0	simple word matching, no AI	a keyword list	0.780	0.730
B1	one standard note, the same for every app	a generic template, never tailored	0.927	0.804
B2	a person writes the note by hand (P2 with ChatGPT)	a human manager	0.858	0.861
B3-k	XLM-R trained on the same examples	a waiter who memorised the menu	0.909	0.833
B4	4 notes chosen at random (average of 5)	a manager choosing by luck	0.959	0.878
A1	our original agent	our automatic manager	0.958	0.841
A2	**our improved agent**	**our automatic manager, better search**	**0.974**	**0.898**

All are development scores on 100 practice requests per project; the final reported scores come from the locked test data. Two honest points: under the plan's full rule (which also limits the drop in entity F1), the original agent A1 fails against B1 on P2, while A2 passes on both projects; and A2 was designed after seeing these development results, so only the test data can confirm it.

14. Is it innovative? An honest answer

What the project can fairly claim:

1. **The whole design process is automated**: description in, tested and exported pipeline out, with no human configuring anything.
2. **It is tested rigorously** against a generic note, a person, random search and the published model, which many similar projects skip.
3. **A real finding:** letting the AI simply pick designs (A1) was no better than random; structuring how it searches (A2) found the best design while testing only 4 of 12.

What it cannot claim: a new kind of pipeline, or an AI that invents designs. The final pipeline is always one of the 12 designs, customised with the project's intents, descriptions and examples. Choosing from a fixed set is a modest contribution; related work such as DSPy already tunes prompts automatically.

15. Why it chooses from 12 designs, and what comes next

Why a fixed set of 12 (the project plan chose this on purpose):

- **Safety:** the AI never writes or runs new code.
- **Proof:** with fixed options we can prove whether the agent is smart, by comparing it with random choices, a person and trying all 12. With unlimited options there is no fair way to measure that.
- **A small AI:** qwen 3B already invents designs that do not exist, writes some poor descriptions and gives some wrong advice; with complete freedom it would likely build worse pipelines.
- **Too little data to judge:** 100 practice sentences are too few to trust an open-ended design.

What comes next: A3, "choose, then refine" (decision 45). After the current work, the agent will start from the best of the 12 and then **rewrite its own note** for the project, using its mistakes: write its own instructions, rewrite descriptions it keeps getting wrong, add rules against extracting extra words, change how many examples it shows. Each change is tested and kept only if the score improves. It still never writes code, so it stays safe and measurable, but the pipeline becomes personalised to the project. A rule for keeping or dropping A3 will be fixed before building it, and A1, A2 and A3 will all be reported on the locked test data.

	Choose from 12 (A1/A2, now)	**Choose, then refine (A3, next)**	**Fully free design (future work)**
Personalised to the project	partly	**yes**	yes
Safe: no AI-written code runs	yes	**yes**	no
Can we prove it is better?	yes	**yes**	hard
Works with a small 3B model?	yes	**probably**	unlikely

Fully free design, where the agent builds any pipeline from scratch, is recorded as future work, for the reasons above.

16. What to say: the whole system in a few sentences

> "An app needs an NLP pipeline to turn what users type into an intent and its details. You can build one by training a model such as XLM-R on examples, or by giving a ready-made AI such as qwen a well-written prompt. Our framework automates the second way: from a description of the app, it writes descriptions of the intents, chooses which of 12 prompt designs to try, tests them on practice requests, keeps the best and saves it for the app. qwen never changes; we only choose the best prompt. We compared it with a generic prompt, a person, random choices and a trained XLM-R, all on the same requests with the same scorer. The improved agent found the best design on both projects while testing only 4 of 12. Next we plan to let it refine the prompt from its own mistakes."

For each question your teacher might ask, the matching section here: what an NLP pipeline is (1), intents versus designs (2), why we need the project (3), what XLM-R is (5), what qwen does (6), what the prompt contains (7), what the agent does (9), why it is not training (11), how the comparison works (12), whether it is innovative (14), why only 12 designs (15).